

THUMB INSTRUCTION REFERENCE

13 Instructions from Real Disassembly

Every distinct Thumb mnemonic in `rec_sum` / `main` / `Reset_Handler` — simplest to most complex

```
arm-none-eabi-objdump -S · main.o & firmware.elf
```

MOV / MOVS

Copy a value into a register — from an immediate or another register

FROM THE LISTING

```
30: movs r3, #0
34: movs r0, #10
1c: mov  r2, r0
```

movs loads an 8-bit immediate and updates the N/Z flags. Plain mov (the “high-register move” encoding) copies register-to-register without touching flags — used here right after a compare, so the flags survive.

WHAT IT DOES

- The simplest data-processing instruction — no arithmetic, just a copy.
- MOVS Rd, #imm8: 16-bit encoding, low registers only, sets flags (S = “sets flags”).
- MOV Rd, Rm (hi-register form): works on any register r0–r15, including sp/pc, and never sets flags — this is why the compiler picks it for “quiet” copies like mov sp, r7.

ADD / ADDS

Add two values; used for arithmetic and for stack-frame bookkeeping

FROM THE LISTING

```
4: add r7, sp, #0
20: add r3, r2
24: adds r7, #8
```

add r7, sp, #0 sets up the frame pointer (SP-relative add, no flags). adds r7, #8 tears the frame back down and does set flags. add r3, r2 is plain arithmetic (a + b) using the hi-register form.

WHAT IT DOES

- Same register/immediate pattern as MOV: the S suffix (ADDS) sets flags, plain ADD (register-to-register hi form or SP-relative) does not.
- Used both for real arithmetic (int c = a + b) and for silently adjusting r7/sp when setting up or tearing down a stack frame.

SUB / SUBS

Subtract; also used to reserve space on the stack

FROM THE LISTING

```
2: sub sp, #8
14: subs r3, #1
```

WHAT IT DOES

- `sub sp, #imm` is how a function prologue allocates its local-variable stack frame — conceptually just “move the stack pointer down”.
- `subs Rd, #imm8` is the arithmetic form the source code actually asked for (`i - 1`), and sets N/Z/C/V like any flag-setting instruction.

sub sp, #8 is a special SP-immediate encoding that reserves 8 bytes of stack for local variables — pure bookkeeping, no flags. subs r3, #1 computes `i-1` for the recursive call and does set flags.

CMP

Compare two values by subtracting them, keeping only the flags

FROM THE LISTING

```
8: ldr r3, [r7, #4]
a: cmp r3, #1
c: dc01 bgt.n 12
```

WHAT IT DOES

- Internally the same subtraction hardware as SUB, but the result is discarded — only N, Z, C, V are updated.
- Always paired with a conditional instruction afterward (here, a conditional branch) — CMP alone changes nothing observable.

Implements if(i <= 1): loads i into r3, compares it with 1. CMP never stores a result — it exists purely to set flags for the branch that follows.

STR

Store a register's value into memory, base register + immediate offset

FROM THE LISTING

```
6: str r0, [r7, #4]
32: str r3, [r7, #4]
```

WHAT IT DOES

- Addressing mode: [base register, #offset] — the address is base + offset, computed once, not modified.
- This is how C local variables that don't fit in registers (or that have their address taken) end up living on the stack.

Both spill a value from a register into its stack slot — the first stores the incoming parameter `i`, the second stores the local variable `sum`, each 4 bytes above the frame pointer `r7`.

LDR

Load a register's value back from memory, same addressing mode as STR

FROM THE LISTING

```
8: ldr r3, [r7, #4]
1e: ldr r3, [r7, #4]
```

WHAT IT DOES

- Same [base, #offset] addressing as STR — LDR and STR are a matched pair, one direction each way.
- A local variable in C is typically stored once and loaded every time it's read — this pattern repeats throughout both functions.

The mirror image of STR: reads the value back out of its stack slot into a register so it can be used in a computation (the comparison, and later the final addition).

LDR (literal)

Load a constant that's too large to fit in an immediate, from a literal pool near the code

FROM THE LISTING

```
8000064: ldr r0, [pc, #40]
@ (8000090 <...>)
8000090: .word 0x080000a6
```

Reset_Handler needs full 32-bit addresses (`_sdata`, `_sdata`, etc.) that can't be encoded as an immediate operand. The assembler stores them as data words right after the code and loads them PC-relative.

WHAT IT DOES

- Uses PC as the base register instead of a general-purpose register — the offset points forward into the instruction stream, at data the assembler placed there (the “literal pool”).
- This is the standard Thumb trick for materializing large constants and addresses, since instructions can't embed a full 32-bit immediate.

LDR.W / STR.W

Wide (32-bit) load/store with post-indexed addressing — auto-increments the base register

FROM THE LISTING

```
800006e: ldr.w r3, [r0], #4
8000072: str.w r3, [r1], #4
```

The FLASH→RAM copy loop in Reset_Handler: load a word from r0, then bump r0 by 4; store a word to r1, then bump r1 by 4 — all in one instruction each.

WHAT IT DOES

- “Post-indexed”: the transfer uses the base register's current value, then the base register is updated afterward — no separate add instruction needed.
- The .W suffix forces the 32-bit Thumb2 encoding, which is what makes post-indexing (and the full r0–r15 register range) available — Thumb1's narrow LDR/STR can't do this.

B / B.n

Unconditional branch — jump to a PC-relative target, always taken

FROM THE LISTING

```
e: ldr r3, [r7, #4]
10: b.n 22 <rec_sum+0x22>
```

Implements the return i; path of the if/else: after loading i, jump straight past the recursive-call code to the function epilogue at offset 0x22.

WHAT IT DOES

- “.n” marks the narrow 16-bit encoding — short-range branches (a few hundred bytes) fit in 16 bits; longer ones would need B.W.
- Also used as the loop-back branch in `copy_data_loop` / `zero_bss_loop`, and as the infinite `b . trap` after `bl main`.

BGT / BGE

Conditional branch — only taken if the flags from a prior CMP satisfy the condition

FROM THE LISTING

```
a: cmp r3, #1
c: bgt.n 12 <rec_sum+0x12>
8000080: bge.n 8000088
```

bgt.n implements if(i <= 1): branch taken only when i > 1 (i.e. skip the base case). bge.n implements the loop-exit test cmp r1,r2 / bge done in both copy loops.

WHAT IT DOES

- Same 16-bit branch encoding as B, but gated by a condition code (GT = signed greater-than, GE = signed greater-or-equal) evaluated from N/Z/C/V.
- This is how Thumb2 compiles every if/while/for condition when the body is too large to fit in an IT block — compare, then branch around.

PUSH / POP

Transfer a list of registers to/from the stack in a single instruction

FROM THE LISTING

```
0: push {r7, lr}
28: pop  {r7, pc}
8000040: push {r7}
800005e: pop  {r7}
```

rec_sum saves r7 and lr on entry (it calls itself, clobbering lr) and restores both on exit — popping straight into pc performs the return. main() in firmware.elf never calls another function, so it only pushes/pops r7, leaving lr untouched.

WHAT IT DOES

- One instruction moves an entire register list — far more compact than one push/pop per register.
- `pop {..., pc}` is a common return idiom: it restores registers and returns in the same instruction, no separate branch needed.
- Whether `lr` appears in the list depends entirely on whether the function calls anything else (compare `rec_sum` vs. `this main`).

BL

Branch with Link — call a function, saving the return address into LR

FROM THE LISTING

```
18: bl 0 <rec_sum>
36: bl 0 <rec_sum>
8000088: bl 8000040 <main>
```

WHAT IT DOES

- The only “call” instruction in Thumb — always a 32-bit encoding, since it needs enough bits for a large PC-relative call offset.
- Because BL overwrites LR, any function that itself calls another function (like `rec_sum`, calling itself) must PUSH/POP {lr} to avoid losing its own return address.

rec_sum calls itself recursively; main calls rec_sum(10); Reset_Handler calls main — three different call sites, same instruction. Each one overwrites LR with the address right after the BL.

BX

Branch and Exchange — jump to an address held in a register

FROM THE LISTING

```
8000060: bx lr
```

firmware.elf's main() is a leaf function — it never calls anything else, so LR still holds its original caller's return address untouched. It restores r7, then returns directly via bx lr instead of popping pc.

WHAT IT DOES

- “Exchange” refers to switching between ARM and Thumb instruction sets based on bit 0 of the target address — on Cortex-M3 (Thumb-only) that just means “branch there”.
- The two ways to return — `pop {..., pc}` vs. `bx lr` — both appear in this listing, depending on whether the function pushed `lr` onto the stack in the first place.

SUMMARY

Four Levels, Thirteen Instructions

Level 1 — Data Movement

MOV/MOVS · ADD/ADDS · SUB/SUBS · CMP

Level 2 — Memory Access

STR · LDR · LDR (literal) · LDR.W/STR.W

Level 3 — Control Flow

B/B.n · BGT/BGE (Bcond) · PUSH/POP

Level 4 — Subroutine Linkage

BL · BX